

AI/ML intern DAY 4

Task 1 - Improved Pose Detection System

LOYA PRANAY (pranay8679@gmail.com)

Date: 14-05-2026

1. Objective

The objective of this task was to improve the accuracy and performance of the real-time human pose detection system using MediaPipe and OpenCV.

The system was designed to:

- improve landmark tracking accuracy
- enhance skeleton visualization
- improve real-time body tracking
- test pose estimation under different body positions

The task also helped in understanding advanced computer vision workflows and AI-based pose estimation systems.

2. Technologies and Platforms Used

Technology / Platform	Purpose
Python	Core programming language used for AI and computer vision implementation
OpenCV	Real-time webcam access and image/video processing
MediaPipe	Human pose detection and body landmark tracking
TensorFlow	AI and deep learning framework
Visual Studio Code	Development and debugging environment
Google Colab	Cloud-based AI testing platform
GitHub	Project management and version control

3. Understanding Improved Pose Detection

Introduction

Human pose detection is a computer vision and Artificial Intelligence technique used to detect and track body landmarks from images and live video streams.

The improved pose detection system was developed to provide:

- smoother landmark tracking
- better body detection accuracy
- improved skeleton visualization
- enhanced real-time performance

The system detects important body landmarks such as:

- shoulders
- elbows
- wrists
- hips
- knees
- ankles

These landmarks are connected together to create a digital skeletal structure of the human body.

Applications

Improved pose detection systems are used in:

- fitness tracking
 - sports analysis
 - healthcare monitoring
 - gaming systems
 - motion capture
 - virtual try-on systems
 - AI fashion technology
-

4. Workflow of the Pose Detection System

The workflow of the improved pose detection system includes the following steps:

Step 1 — Webcam Access

OpenCV accesses the webcam and captures real-time video frames.

Step 2 — Image Preprocessing

The captured frames are resized and converted from BGR format to RGB format for MediaPipe processing.

Step 3 — Pose Detection

MediaPipe processes the RGB image and detects body landmarks using AI pose estimation models.

Step 4 — Skeleton Tracking

Detected landmarks are connected together to form the body skeleton.

Step 5 — Real-Time Visualization

The processed video with body landmarks and skeleton tracking is displayed in real time.

5. Code Implementation

Improved Pose Detection Program

```
import cv2
import mediapipe as mp

# Initialize MediaPipe Pose
mp_pose = mp.solutions.pose

pose = mp_pose.Pose(
    static_image_mode=False,
    model_complexity=2,
    smooth_landmarks=True,
    enable_segmentation=True,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)
```

```
# Drawing utilities
mp_draw = mp.solutions.drawing_utils

# Open webcam
cap = cv2.VideoCapture(0)

while True:
    success, frame = cap.read()

    if not success:
        break

    # Resize frame
    frame = cv2.resize(frame, (960, 720))

    # Convert BGR to RGB
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

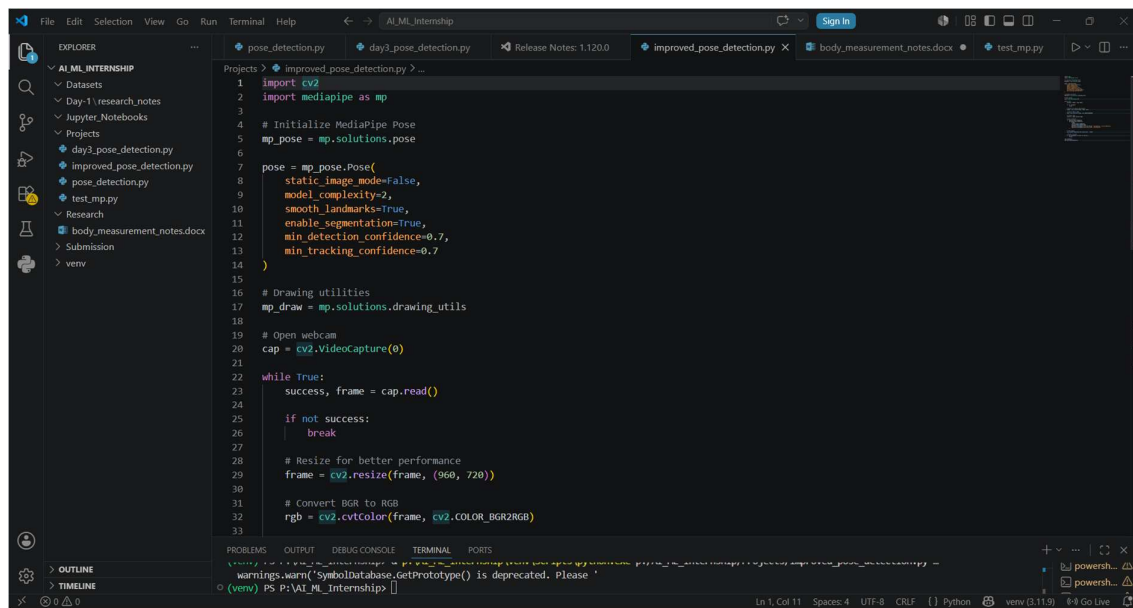
    # Process pose
    results = pose.process(rgb)

    # Draw landmarks
    if results.pose_landmarks:
        mp_draw.draw_landmarks(
            frame,
            results.pose_landmarks,
            mp_pose.POSE_CONNECTIONS,
            mp_draw.DrawingSpec(color=(0,255,0), thickness=2, circle_radius=2),
            mp_draw.DrawingSpec(color=(0,0,255), thickness=2)
        )

    # Display output
    cv2.imshow("Improved Pose Detection", frame)

    # Exit condition
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



The screenshot shows a Visual Studio Code editor window with a project named 'AI ML Internship'. The Explorer sidebar on the left lists files including 'pose_detection.py', 'day3_pose_detection.py', 'improved_pose_detection.py', 'test_mp.py', and 'body_measurement_notes.docx'. The main editor area displays the code for 'improved_pose_detection.py'. The code imports 'cv2' and 'mediapipe as mp', initializes a MediaPipe Pose model, opens a webcam, and enters a loop to read frames, detect poses, and draw landmarks. The terminal at the bottom shows a warning about a deprecated function and the current directory path 'P:\AI ML Internship'.

```
1 import cv2
2 import mediapipe as mp
3
4 # Initialize MediaPipe Pose
5 mp_pose = mp.solutions.pose
6
7 pose = mp_pose.Pose(
8     static_image_mode=False,
9     model_complexity=2,
10    smooth_landmarks=True,
11    enable_segmentation=True,
12    min_detection_confidence=0.7,
13    min_tracking_confidence=0.7
14 )
15
16 # Drawing utilities
17 mp_draw = mp.solutions.drawing_utils
18
19 # Open webcam
20 cap = cv2.VideoCapture(0)
21
22 while True:
23     success, frame = cap.read()
24
25     if not success:
26         break
27
28     # Resize for better performance
29     frame = cv2.resize(frame, (960, 720))
30
31     # Convert BGR to RGB
32     rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

6. Output and Results

The improved pose detection system successfully:

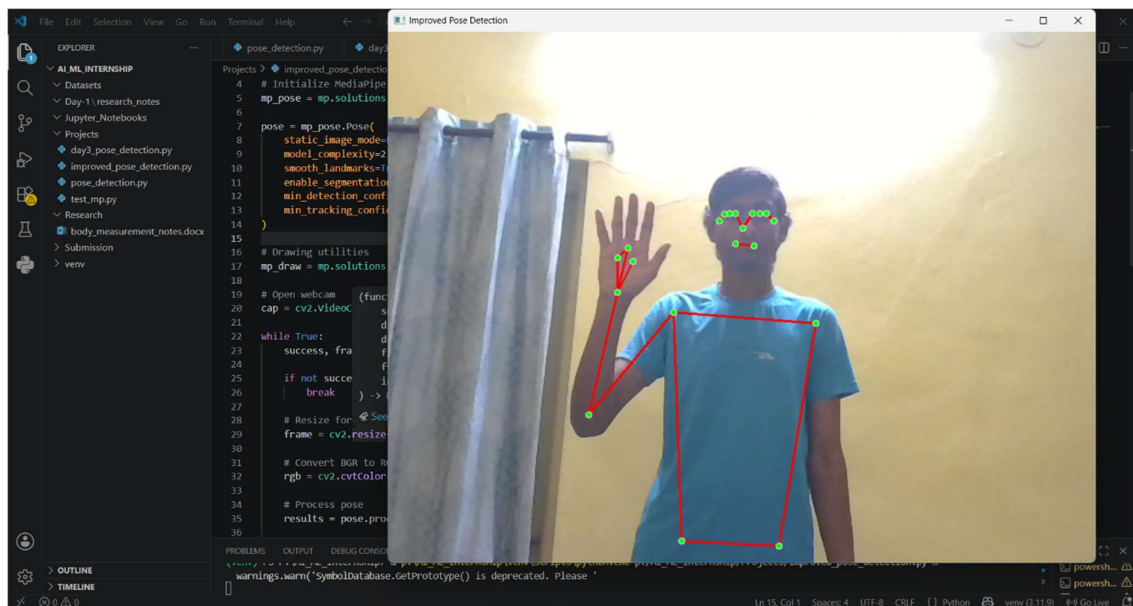
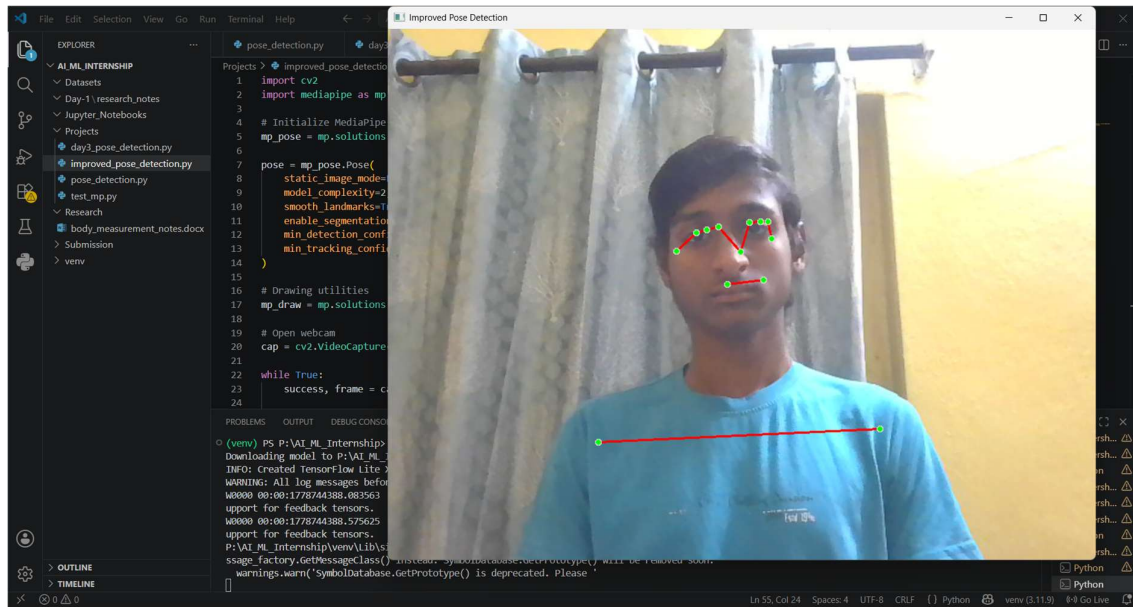
- detected body landmarks
- tracked body movements in real time
- improved skeleton visualization
- enhanced landmark smoothness
- improved tracking accuracy

The system was tested using:

- standing posture
- side posture
- hand movement

The implementation demonstrated the practical use of:

- Artificial Intelligence
- Computer Vision
- Pose Estimation
- Real-Time Human Tracking



7. Accuracy Testing of Pose Detection System

Objective

The objective of this testing phase was to evaluate the accuracy and stability of the improved pose detection system under different body positions and movements.

The system was tested using:

- standing posture
- side posture
- hand movement
- walking movement
- partial body visibility

The improved pose detection system successfully:

- tracked body landmarks smoothly
- reduced landmark flickering
- improved skeleton visualization
- maintained stable body tracking
- improved movement detection accuracy

The system performed best when:

- the full body was visible
- lighting conditions were good
- the camera angle was clear

The pose detection accuracy slightly reduced during:

- fast body movement
- poor lighting conditions
- partial body visibility

8. Improvements Made in the System

The following improvements were implemented compared to the previous pose detection system:

Previous System	Improved System
Basic landmark tracking	Enhanced tracking accuracy
Simple visualization	Improved skeleton visualization
Lower detection confidence	Higher confidence threshold
Basic tracking	Smooth landmark tracking
Standard processing	Better real-time performance

9. Challenges Faced and Solutions

Challenge Faced	Solution Applied
Webcam lag issues	Reduced unnecessary processing
Landmark flickering	Enabled smooth landmark tracking
Detection instability	Increased confidence thresholds
Lighting issues	Tested under better lighting conditions
Pose tracking delays	Optimized frame resizing

10. Learning Outcomes

During this task, the following concepts and skills were learned:

- advanced pose estimation
- real-time body tracking
- skeleton visualization
- landmark smoothing
- AI-based body analysis
- OpenCV image processing
- MediaPipe configuration
- webcam processing workflows

11. Conclusion

This task provided practical exposure to advanced human pose detection systems using Artificial Intelligence and computer vision technologies.

The improved pose detection system successfully enhanced:

- body tracking accuracy
- landmark smoothness
- skeleton visualization
- real-time processing performance

The task also improved understanding of:

- AI pose estimation
- computer vision workflows
- real-time webcam processing
- body landmark tracking systems

The project established a stronger foundation for future AI and computer vision applications in fashion technology and virtual try-on systems.